🧠 What is Polymorphism?

Polymorphism means *many forms*.
It allows **the same function or method name to behave differently** based on the object or data type that it is acting upon.

In simple words:

Same method name → different behaviors → depending on the object.

🔹 Example 1: Polymorphism with Built-in Functions

Python functions like len() work differently with different data types.

```
# Example of polymorphism using built-in function

print(len("Python"))     # Works on a string -> Output: 6

print(len([10, 20, 30])) # Works on a list -> Output: 3

print(len({'a': 1, 'b': 2}))  # Works on a dictionary -> Output: 2
```

✅ **Same function (len)**, but behavior changes depending on the object type.

🔹 Example 2: Polymorphism with Class Methods

Let's say we have two different classes, both having a sound() method.

```
class Dog:
    def sound(self):
        return "Woof!"


class Cat:
    def sound(self):
        return "Meow!"
```

Now both classes have a method with the same name but **different behavior**.

```
# Polymorphism in action
```

```
for animal in (Dog(), Cat()):
    print(animal.sound())
```

Output:

```
Woof!
Meow!
```

✅ Here, sound() behaves differently depending on whether the object is a Dog or a Cat.

🔹 **Example 3: Polymorphism with Inheritance (Method Overriding)**

In inheritance, a child class can override the parent's method.

```
class Bird:
    def fly(self):
        return "Most birds can fly"

class Penguin(Bird):
    def fly(self):
        return "Penguins cannot fly"
bird = Bird()
penguin = Penguin()

print(bird.fly())      # Output: Most birds can fly
print(penguin.fly())   # Output: Penguins cannot fly
```

✅ The same method fly() has **different behaviors** depending on the class.

🔹 **Example 4: Polymorphism with Functions and Objects**

You can write a function that works with any object that implements a certain method.

```
class Duck:
```

```
    def swim(self):
        print("Duck swims in the pond")

class Fish:
    def swim(self):
        print("Fish swims in the sea")

def make_it_swim(animal):
    animal.swim()  # No matter what 'animal' is, as long as it has 'swim'

duck = Duck()
fish = Fish()

make_it_swim(duck)
make_it_swim(fish)
```

Output:

```
Duck swims in the pond
Fish swims in the sea
```

✅ This is **Duck Typing** in Python —
"If it looks like a duck and quacks like a duck, it's treated like a duck."

🔹 **Example 5: Operator Overloading (Another form of Polymorphism)**

Operators like +, -, *, etc., work differently depending on operands.

```
print(5 + 10)       # Adds two numbers -> Output: 15
print("Hi " + "There")  # Concatenates two strings -> Output: Hi There
print([1, 2] + [3, 4])  # Joins two lists -> Output: [1, 2, 3, 4]
```

✅ The + operator is **overloaded** to work with different types.

🔹 **Example 6: Custom Operator Overloading**

You can define how operators behave for your custom classes using **magic methods** like __add__.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(2, 3)
p2 = Point(4, 1)
print(p1 + p2)  # Output: (6, 4)
```

✅ Here, we **overloaded the + operator** for custom addition of Point objects.

🧩 **Summary Table**

Type of Polymorphism	Example / Explanation
Built-in function polymorphism	len("abc"), len([1,2,3])
Method overriding (Inheritance)	Child overrides parent method

Type of Polymorphism	Example / Explanation
Duck typing	Any object with required method works
Operator overloading	Custom meaning for +, -, etc.